

TutorGen

A grounded AI teaching assistant for a creative coding course

CLIENT	YEAR
Personal project	2026–present

TutorGen is an AI teaching assistant I built in 2026 for my own creative coding course. Students watch the lesson video and a chat panel side by side. The tutor is grounded entirely in the transcripts of the class recordings: it identifies which lessons are relevant to a question, answers in the voice of a good teaching assistant — guiding students toward a solution rather than handing it over — and cites specific moments in the video that seek the player when clicked. When it genuinely does not know, it says so and logs the question for me to review.

Repository: github.com/andrew-atkinson/creative-coding-assistant

(<https://github.com/andrew-atkinson/creative-coding-assistant>) · **Live demo:**

teaching.andrewatkinson.net (<https://teaching.andrewatkinson.net/c/creative-coding-101>)

The Premise

The reflexive worry about AI in a coding course is that it becomes an answer machine: a student pastes an error, receives working code, and learns nothing. TutorGen takes the opposite stance. Its persona is a teaching assistant, and a good TA does not solve the problem for you — it asks what you have tried and points you at the part of the lesson that addresses it. That system prompt is doing pedagogical work.

Grounding is the second premise. A general model will answer a p5.js question with Processing syntax because it optimizes for fluent text, not for what this course taught. TutorGen only knows what the course knows: its entire knowledge is the lesson transcripts, so its answers stay consistent with the tools, order, and vocabulary the students actually encountered. The third premise is honest ignorance — when it cannot ground an answer, it refuses rather than improvising, and the unanswered question becomes a curriculum signal about where the course leaves students short.

How it works

The retrieval design is deliberately simple. There is no vector search: a small, fast router model selects the relevant lesson or lessons for a question, and the answering model then receives the *full* transcript of each in its context, so it sees complete, coherent lessons rather than disconnected chunks. Citations are the other piece of pedagogical machinery — as the answer streams in, each claim is marked with a timestamp and video, rendered as a clickable chip that seeks the player to that moment. Hallucinated references that point at no real lesson are silently dropped. The effect is that the tutor keeps sending students back to the source rather than replacing it.
